

MILESTONE REPORT

SnapChef

A stick-on smart-fridge companion that recognizes groceries, tracks your inventory, and suggests recipes — on two ESP32-S3 boards.

Team

Team SnapChef

Members

Eric Meng
Iris Yang
Louis Liu

Repository

github.com/IrisYzh/SnapChef

PROBLEM

Food gets wasted because no one tracks the fridge

People forget what they already own, buy duplicates, and let produce rot out of sight. Existing “smart fridges” are expensive, built-in, and can’t be added to the fridge you already have.

- No lightweight way to log items in/out without an app or typing.
- Produce is hard to track — no barcode, many varieties.
- Receipts hold the data but get thrown away.
- Users want recipe ideas from what they own before it spoils.

Intended users

- Home cooks & busy households who shop weekly and cook from fresh produce.
- Students / shared apartments who can’t justify a built-in smart fridge.
- People who want zero-friction input — hold an item to a camera or snap a receipt.

A peel-and-stick device that turns any fridge into a tracked one.

SOLUTION

A two-board fridge sticker: camera + touchscreen

A sensor/compute board (camera, distance sensor, Wi-Fi) talks over ESP-NOW to a 4.3" touch display that owns the UI and inventory. Heavy AI runs in the cloud.



Put in

Hold a fruit/veg to the camera, or snap a grocery receipt. Items are recognized and merged into your fridge inventory.



Take out + recipes

Scan an item to remove it, or pick from the list. SnapChef suggests recipes from what's left and shows which items each uses.



My Fridge

Browse inventory sorted by name & time added; remove by tap. Proximity wake lights the screen as you approach.

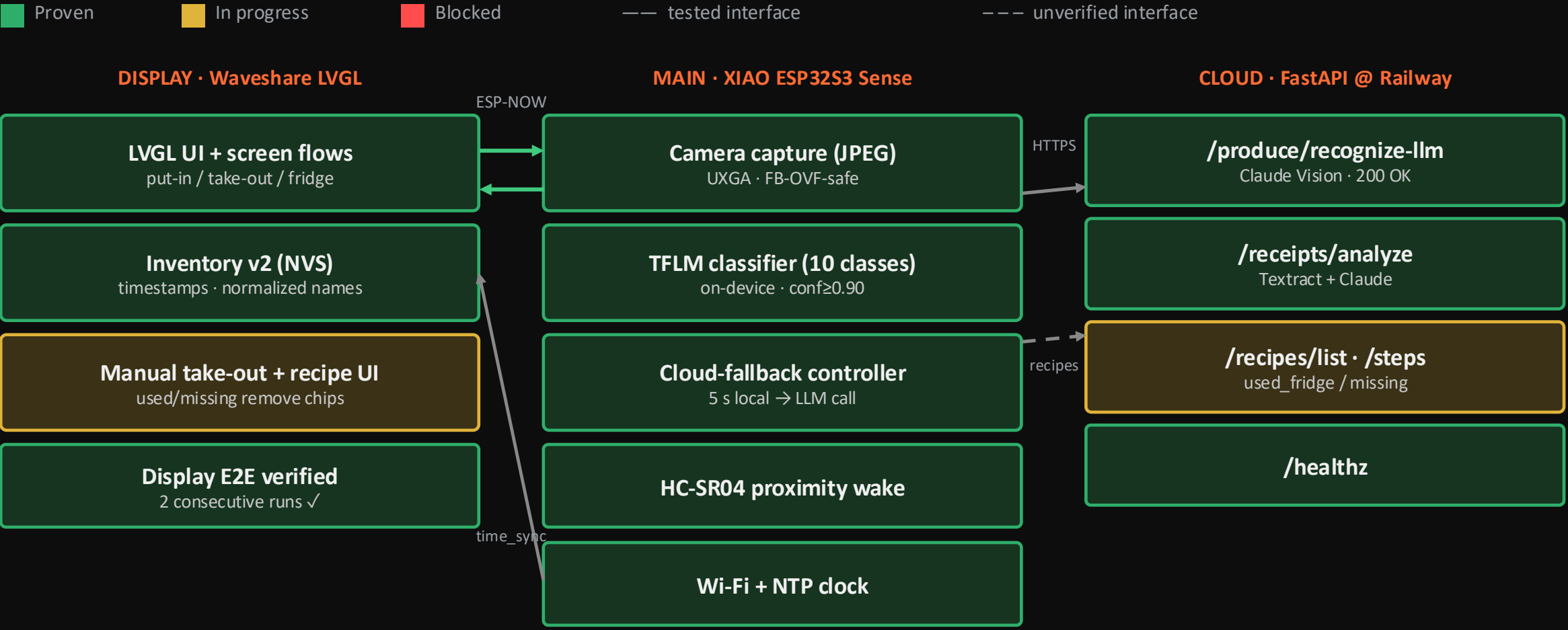
Key features

- On-device ML (TFLM, 10 produce classes) — instant, offline.
- Cloud LLM-vision fallback for any produce outside those classes.
- Receipt OCR → structured grocery list with refrigeration tags.
- LLM recipe suggestions from current inventory.

Hardware & stack

- Main: Seeed XIAO ESP32S3 Sense (camera + HC-SR04 + Wi-Fi).
- Display: Waveshare ESP32-S3-Touch-LCD-4.3 (LVGL UI).
- Link: ESP-NOW peer-to-peer between boards.
- Backend: FastAPI on Railway + Claude Vision / Baidu / Textract.

System diagram — color = maturity · line = interface



Change since Milestone 1: BLE link → ESP-NOW; on-device-only recognition now has a cloud LLM fallback; inventory gained timestamps, normalized names, manual take-out and recipe used/missing UI.

Two measurable improvements

① Recognition coverage & accuracy

Metric	Before (M2)	After
Recognizable items	10 classes	arbitrary
Plastic / hand-held / glare	Uncertain 0.37	correct 0.95
Lock threshold	0.80	0.90
Out-of-class result	"Unknown"	named (LLM)

② Latency & capture quality

Metric	Before (M2)	After
Local window → fallback	20 s	5 s
Cloud recognize latency	—	1.2–2.7 s
Capture quality (lower=better)	q12	q6
Camera overflow (FB-OVF)	on q<init	eliminated

Qualitative analysis

- **Coverage:** the on-device model knew only 10 classes and rejected the rest; the LLM fallback makes recognition open-vocabulary while keeping the fast offline path for common items.
- **Robustness:** on hard inputs Baidu scored 0.37→Uncertain while Claude Vision returned the correct item at 0.95; the 0.90 lock also cuts false on-device locks.
- **Responsiveness:** cutting the local window 20s→5s reaches a cloud answer sooner for off-list items, and higher-quality capture improves OCR + produce accuracy with no overflow.

End-to-end runs & cross-condition testing

E2E pipeline — 2 consecutive runs (full device)

camera → ESP-NOW → cloud → result shown on the display

Run	Input	Latency	Result
1	OOD corn (off-list)	2.1 s	Corn 0.95 ✓
2	bell pepper	1.3 s	Pepper 0.85 ✓

Display device now built & running — both runs completed the full pipeline and rendered on-screen.

Cross-condition results

Condition	Input	Outcome
Real produce	ripe tomato	Tomato 0.88 ✓
OOD produce	corn (off-list)	Corn 0.95 ✓
Hand-held / glare	bell pepper	Pepper 0.85 ✓
Non-produce	water bottle	Okra 0.75 ✗
Diff device / user	2 boards, 2 IPs	both 200 ✓

OOD = outside the 10 on-device classes — handled by the LLM fallback. Names normalized: Red Pepper → Pepper.

Status summary

Proven: full device E2E (camera→cloud→display), ESP-NOW link, produce-LLM & receipt endpoints, on-device TFLM, inventory v2.

In progress: recipe used/missing UI & /recipes endpoints. **Known gap:** non-produce over-recognition → backend Uncertain rules + vocab.

BUDGET UPDATE

Spend to date

Unit prices are typical retail estimates — **replace with actual receipts**

Item	Qty	Unit (est.)	Subtotal	Status
Seeed XIAO ESP32S3 Sense (incl. camera)	1	\$23.99	\$23.99	purchased
Waveshare ESP32-S3-Touch-LCD-4.3	1	\$45.99	\$45.99	purchased
HC-SR04 ultrasonic sensor	1	\$3.50	\$3.50	purchased
Wires, resistor divider, misc.	1	\$5.00	\$5.00	purchased
Railway hosting (backend)	~1 mo	\$5.00	\$5.00	recurring
Claude / Baidu / Textract API	usage	~\$0–10	~\$5.00	usage
Total spent (est.)			≈ \$88.48	

Total budget

\$200

Spent

≈ \$88

Remaining

\$118

Fill in your allocated total budget; remaining = total – spent.